PEDIA CENTRE FRONTEND · END-TO-END TESTING

How Playwright was used to test this application

A full walkthrough of installation, file layout, the mock backend, session seeding, every spec file, and the exact commands that run the suite.

Angular 21 app on :4200

Playwright 1.65

29 browser tests

API mocked — no FastAPI needed

Project: pedia-centre-frontend

Prepared as a testing process guide

Colour PDF walkthrough

1. What Playwright is doing here

Playwright is a browser automation library. For this project it is not a unit tester (that is Angular / Vitest). It opens a real Chrome window (headless or headed), loads the Angular UI at `http://localhost:4200`, clicks buttons, fills forms, follows routes, and checks that the screen shows the right text and URLs.

The key design choice

The Angular app still thinks it is talking to FastAPI at `http://127.0.0.1:8000/api/v1`. Playwright intercepts every matching request before it leaves the browser and returns fake JSON. That is why the suite can run without the Python backend, a database, or real users.

1. Config
playwright.config.ts starts Angular with `npm start` and waits for :4200.

→

2. Browser
Chrome launches. Each test gets a fresh page and empty storage.

→

3. Mock
`mockBackend(page)` stubs `**/api/v1/**`.

→

4. Act
Goto, click, type, submit — like a real parent, doctor, or admin.

→

5. Assert
expect() checks URL, headings, errors, and success messages.

2. Why this shape of tests

What we are proving

- Routing and role guards actually block the wrong role
- Login / register forms validate and navigate correctly
- Each role area (guardian, doctor, admin) can complete its main jobs
- The UI binds mocked API data onto the screen

What we are not proving

- Real FastAPI business rules or SQL
- True JWT signing / expiry on the server
- Network failure of a live clinic API
- Pixel-perfect CSS across every device (Chromium only)

Session is stored in the browser as `localStorage.pc_session`. AuthService reads that key. Tests either log in through the UI, or write the session directly with `seedSession()` so later pages can skip the login form.

3. Installation — exact commands used

Playwright was added as a **devDependency** of the Angular repo, not as a separate project.

STEP A · PACKAGE

```
npm install -D @playwright/test
```

This wrote `"@playwright/test": "^1.65.0"` into `package.json` and locked it in `package-lock.json`.

STEP B · BROWSER BINARY

```
npx playwright install chrome
```

The config uses `channel: 'chrome'`, so tests drive the installed Google Chrome instead of Playwright's bundled Chromium. If Chrome is missing on a machine, either install Chrome or switch the config to Playwright Chromium and run:

```
npx playwright install chromium
```

STEP C · NPM SCRIPTS (package.json)

```
"test:e2e": "playwright test",
"test:e2e:ui": "playwright test --ui"
```

STEP D · GITIGNORE

Generated run output must not be committed. These folders were ignored:

```
/test-results/
/playwright-report/
/blob-report/
/playwright/.cache/
/playwright/.auth/
```

Angular's own `npm test` was left alone

That script still runs unit tests. End-to-end is a separate command so a failed Karma/Vitest run cannot be confused with a failed browser flow.

4. Where files were created and why

Playwright's default is a `tests/` folder at the repo root — next to `src/`, not inside Angular components. Specs live under `tests/e2e` because that is what `testDir` points at in the config.

```
pedia-centre-frontend/
├── playwright.config.ts          ← engine: URL, browser, webServer, reporters
├── package.json                  ← @playwright/test + test:e2e scripts
├── src/app/                      ← Angular UI under test (not test code)
└── tests/
    ├── e2e/                      ← only *.spec.ts files are executed
    │   ├── smoke.spec.ts
    │   ├── auth.spec.ts
    │   ├── register.spec.ts
    │   ├── access-control.spec.ts
    │   ├── guardian.spec.ts
    │   ├── doctor.spec.ts
    │   └── admin.spec.ts
    ├── helpers/                  ← shared fake world
    │   ├── fixtures.ts           ← JSON shapes (doctors, children, appointments)
    │   ├── jwt.ts                ← tiny unsigned JWT AuthService can decode
    │   ├── session.ts            ← TEST_USERS + seedSession()
    │   └── mocks.ts              ← page.route('**/api/v1/**')
    └── pages/
        └── login.page.ts         ← page object for the login screen
```

File	Job	Used by
playwright.config.ts	How to run: baseURL, Chrome, start Angular, 1 worker	CLI
tests/helpers/fixtures.ts	Stable fake clinic data (Dr Meera Shah, Aarav, APT-501…)	mocks
tests/helpers/jwt.ts	Builds header.payload.signature so AuthService reads sub + exp	mocks, session
tests/helpers/session.ts	Known users + writes pc_session into localStorage	most specs
tests/helpers/mocks.ts	In-browser fake REST API	every spec
tests/pages/login.page.ts	Reusable clicks for role, identifier, password, Sign in	auth.spec.ts

5. playwright.config.ts — what each setting means

Setting	Value in this repo	Why
testDir	./tests/e2e	Only specs run; helpers/pages are imported, not executed as tests
fullyParallel	false	Flows share mocked state ideas; keep order simple
workers	1	One Chrome at a time; Angular + intercepts stay predictable
retries	2 on CI, 0 locally	Flakes retry in CI only
baseURL	http://localhost:4200	page.goto('/login') hits the Angular dev server
trace / screenshot / video	on failure	Debug a red test without recording every green one
projects	chromium + channel chrome	Desktop Chrome only
webServer.command	npm start	Playwright boots `ng serve` if :4200 is down
reuseExistingServer	true locally	If you already ran ng serve, tests reuse it
reporter	list + html	Terminal progress plus playwright-report/index.html

webServer timeout is 120 seconds

First Angular compile can be slow. Playwright waits until :4200 answers before any spec starts. If the app never boots, every test fails with a server-start error, not a UI assertion.

6. The fake clinic: users, JWT, and API mocks

Test accounts (tests/helpers/session.ts)

Role	Identifier	Password	userId
guardian / parent	9876543210	Parent@123	11
doctor	DOC001	Doctor@123	21
admin	admin	Admin@123	1

A second doctor login exists only in the mock: `DOC-TEMP / Temp@1234` returns `must_change_password: true`.

Fake JWT (tests/helpers/jwt.ts)

AuthService does not verify a server signature. It only needs a three-part token whose payload JSON has `sub` (user id) and `exp` (unix expiry). The helper base64url-encodes a header and payload and appends the dummy string `test-signature`.

How mockBackend works (tests/helpers/mocks.ts)

```
await page.route('**/api/v1/**', async (route) => { ... });
```

Angular HttpClient still posts to `http://127.0.0.1:8000/api/v1/...`. Playwright matches that URL pattern inside the page, reads method + path, and `route.fulfill()`s JSON. Anything unmapped returns 404 with `Unmocked METHOD path` so a missing stub is obvious.

Examples of stubbed calls

- POST /auth/login · parent token or 401
- POST /auth/doctor-login · schedule vs change-password
- POST /registration · 201 + MRN-101
- GET /doctors/available · Meera Shah or empty
- POST /appointments · APT-501
- POST /doctors · staff id DOC099

seedSession shortcut

Opens /login, then:

```
localStorage.setItem(
  'pc_session',
  JSON.stringify({ token, role, userId, name, mustChangePassword })
)
```

Used for guardian / doctor / admin area tests so we do not re-click Sign in every time.

7. Page object: LoginPage

`tests/pages/login.page.ts` wraps the login screen so auth tests speak in user language instead of repeating locators. Locators prefer roles and labels — the same names a screen reader (and a human) sees.

```
await login.goto();
await login.login('Parent', '9876543210', 'Parent@123');
await expect(page).toHaveURL(/\/guardian\/find-doctor/);
```

Only login uses a page object. Other screens are unique enough that the spec files click them directly with `getByRole`, `getByLabel`, and `getByPlaceholder`.

8. The 29 tests, file by file

Spec	Count	What the browser proves
smoke smoke.spec.ts	2	/ and unknown URLs redirect to /login. Heading “Pedia Centre” and Parent / Doctor / Admin buttons exist — the app is not a blank shell.
auth auth.spec.ts	6	Parent mobile validation; bad password stays on login; parent → find-doctor; doctor → schedule; temp doctor → change-password; admin → /admin/doctors. Admin does not see “Create an account”.
register register.spec.ts	4	Login ↔ register links; empty form errors; add/remove Child 2; happy path shows “Registration successful” and MRN-101 from the mock.
guards access-control.spec.ts	4	Logged-out users bounced from all three areas. Parent cannot open doctor/admin. Doctor cannot open guardian/admin. Log out clears access.
guardian guardian.spec.ts	4	Find doctor + reset filters; book slot 10:00 and see APT-501; list + cancel appointment; treatment history for Aarav shows Viral fever / Paracetamol.
doctor doctor.spec.ts	5	Schedule → history; search patients down to Anaya; add treatment record; change password; forced password change hides clinical nav.
admin admin.spec.ts	4	Search doctors; availability link redirects to list; create doctor shows DOC099; assign doctor to Aarav; create user + reset password.

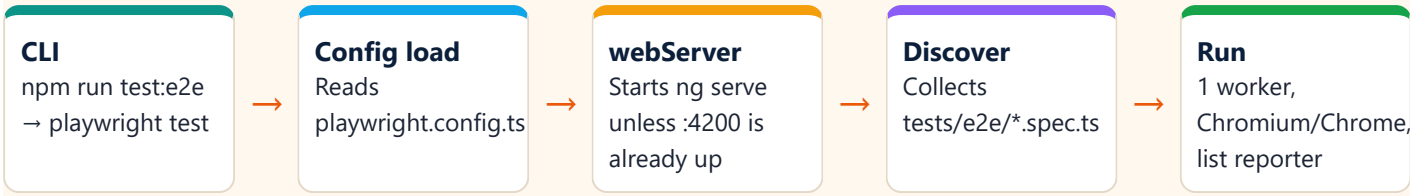
9. One test, second by second

Example: parent books an appointment (`guardian.spec.ts`).

1. **beforeEach** — `mockBackend(page)` then `seedSession(page, 'guardian')` .
2. **goto** — `/guardian/book/21` . Role guard sees pc_session.role = guardian, so the page loads.
3. Angular calls GET doctor 21 and GET children. Playwright returns Dr Meera Shah and Aarav / Anaya.
4. Test selects child 101, fills a date 7 days ahead. App requests slots. Mock returns 10:00, 10:15, 11:00.
5. Test clicks 10:00, types “Fever”, clicks Confirm booking.
6. POST /appointments is intercepted → 201 with APT-501. UI shows “Appointment booked”.
7. Click “View my appointments” → URL is /guardian/my-appointments.

If the mock is wrong, the UI test fails
That is useful. A 404 Unmocked path means the Angular service called an endpoint the fake API does not know. A missing heading means the component did not render the JSON we did return.

10. Runtime: what happens when you run the suite



Each test gets an isolated browser context: empty cookies, empty localStorage, unless the test writes them. After the last test, Playwright writes `playwright-report/`. Failures also land under `test-results/` (screenshot, video, trace.zip).

11. Commands to run and inspect

FULL SUITE

```
npm run test:e2e
# same as:
npx playwright test
```

INTERACTIVE UI MODE

```
npm run test:e2e:ui
# watch tests, pick one spec, time-travel through actions
```

ONE FILE OR ONE TITLE

```
npx playwright test tests/e2e/auth.spec.ts
npx playwright test tests/e2e/guardian.spec.ts -g "books an appointment"
```

SEE THE BROWSER (NOT HEADLESS)

```
npx playwright test --headed
npx playwright test --debug
```

LAST HTML REPORT

```
npx playwright show-report
```

OPEN A FAILURE TRACE

```
npx playwright show-trace test-results/**/trace.zip
```

Local convenience

Keep `ng serve` running in another terminal. Config reuseExistingServer is true, so Playwright will not start a second Angular process.

CI note

On CI, reuseExistingServer is false, retries = 2, forbidOnly is on. The job must install Chrome (or switch off channel: 'chrome') and run npm run test:e2e.

12. How this maps onto the Angular routes

App route	Guard	Covered mainly by
/login, /register	public	smoke, auth, register
/guardian/find-doctor	roleGuard('guardian')	auth, guardian, access-control
/guardian/book/:doctorId	guardian	guardian.spec.ts
/guardian/my-appointments	guardian	guardian.spec.ts
/guardian/treatment-history	guardian	guardian.spec.ts
/doctor/schedule, /patients, history, add-record	doctor + passwordChangedGuard	doctor.spec.ts, auth
/doctor/change-password	doctor	auth + doctor
/admin/doctors, /doctors/new, /patients, /users	roleGuard('admin')	admin.spec.ts, auth
/admin/doctors/:id/availability	redirects to list	admin.spec.ts
unknown path	** → login	smoke.spec.ts

13. Process recap — how Playwright was introduced

1. Installed `@playwright/test` in this Angular repo (devDependency).
2. Installed the Chrome channel Playwright drives.
3. Added `playwright.config.ts` with baseURL :4200 and webServer = npm start.
4. Created `tests/helpers` first: fixtures (the fake clinic), jwt, session, then the big route mock so every later spec could call `mockBackend(page)` in beforeEach.
5. Created a small LoginPage for the three-role sign-in screen.
6. Wrote specs in product order: smoke → auth → register → guards → guardian → doctor → admin.
7. Wired `npm run test:e2e` and `test:e2e:ui`.
8. Ignored report folders in `.gitignore`.

Mental model to keep

Playwright is a remote control for Chrome. Angular is the app. mocks.ts is a pretend FastAPI sitting inside the browser. fixtures.ts is the clinic's sample data. session.ts is a fake already-logged-in cookie jar. Spec files are the scripts that pretend to be Parent, Doctor, or Admin.

14. Quick cheat sheet

Install again on a new machine

```
npm install
npx playwright install chrome
```

Run

```
npm run test:e2e
npm run test:e2e:ui
```

Backend required?

No. API is mocked. Only Node, npm packages, Chrome, and a successful ng `serve` are required.

Where to look when red

Terminal error → screenshot in test-results/ → HTML report → trace viewer. If "Unmocked", extend mocks.ts.

Pedia Centre Frontend · Playwright 1.65 · tests/e2e · 29 tests · Chromium / Google Chrome · mock API at **/api/v1/**